

Introducción a la programación distribuida con MPICH

Joaquín Cuéllar Padilla joaquin.cuellar@uco.es

Abril de 2026

Resumen

En esta práctica vais a introducir en la programación distribuida usando la librería MPICH. Vais a programar en C. Los objetivos son:

- Practicar la programación en C. Uso de memoria, depuración...
- Diseñar e implementar un algoritmo de cálculo que distribuya la carga computacional entre distintos nodos, usando programación distribuida.
- Utilizar para ese fin la librería MPICH, saber leer la documentación oficial, las cabeceras de la librería y buscar ayuda y recursos en la red.
- Defender mediante exposición oral el trabajo desarrollado.

1. Introduciendo la programación distribuida y las tecnologías que emplearéis en esta práctica

1.1. ¿Qué es la programación distribuida?

[wikipedia](#)

Básicamente vamos a decir, que en comparación con un sistema de memoria compartida donde diversos procesadores trabajan compartiendo la misma memoria, en programación distribuida cada procesador tendrá su propia memoria y la información entre las diversas máquinas se transmitirá en forma de mensajes.

Esta tecnología se emplea principalmente en supercomputación, donde se distribuye el cómputo entre distintas máquinas distanciadas geográficamente por sus especiales capacidades de cómputo, por necesidad de realizar accesos a datos locales en distintos lugares...

Podéis observar de forma gráfica la diferencia entre programación distribuida y paralela en la Figura 1.

1.2. ¿Qué tecnologías vamos a emplear?

Esta práctica la vamos a desarrollar **usando C** y la librería **MPICH en su versión 4.0.3**. (imagen de docker propia) ó **3.2**. (imagen de docker en docker hub) Para el despliegue **vamos a emplear**

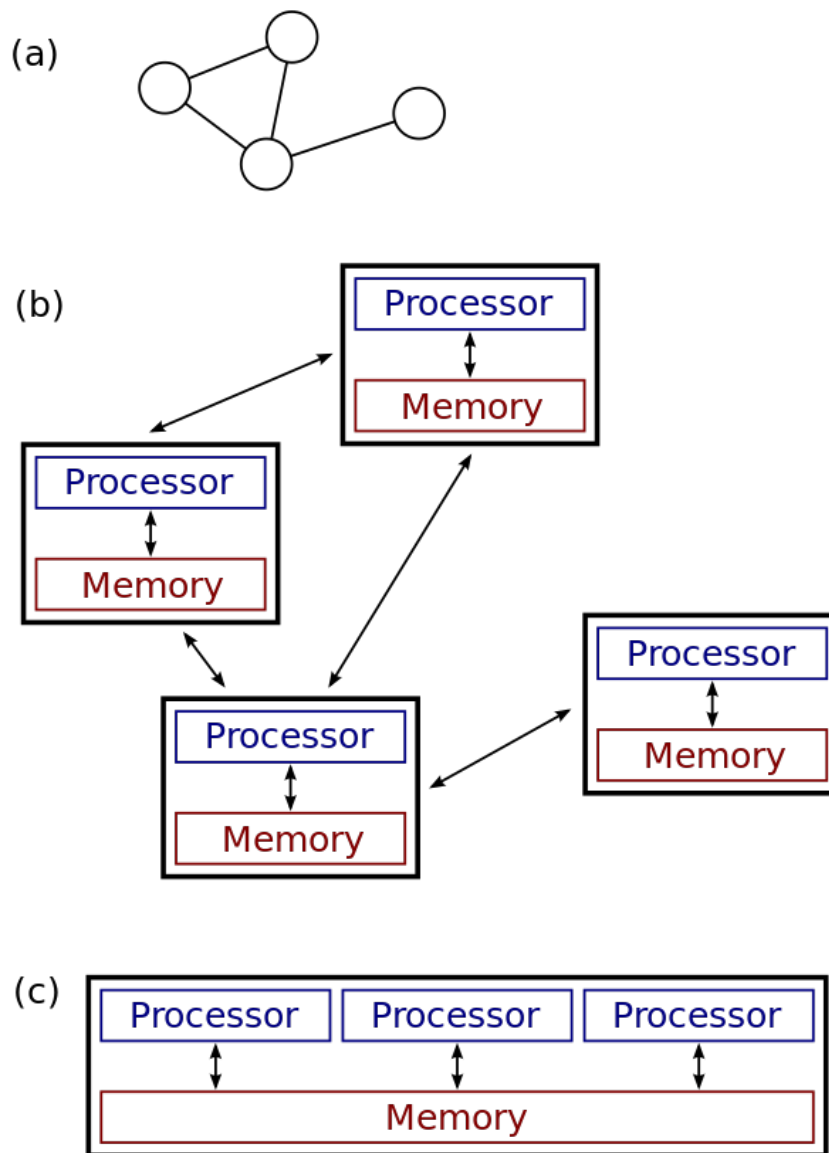


Figura 1: Gráfico comparativo, programación distribuida vs paralela.

docker.

¿Porqué?:

- C es rudimentario pero óptimo a nivel de eficiencia en tiempo de ejecución y recursos, y sigue siendo de lejos el lenguaje más empleado en computación distribuida y sistemas empujados por esta eficiencia y flexibilidad en el empleo de recursos y memoria.
- MPICH es la implementación de MPI que más se asemeja al estándar, es [software libre](#), bien mantenida, multiplataforma y con buena documentación. La vamos a emplear en su versión 4.0.3 y no en la última, porque a partir de la versión 4.1 han incluido cambios que la hacen difícil de compilar con la imagen de docker (configura un Alpine Linux con Musl en vez de Glibc) que empleamos.
- Docker es una tecnología básica a día de hoy, y nos va a permitir configurarnos un clúster de manera sencilla en el mismo equipo de trabajo, además limitando los recursos de cada

nodo de forma que podamos simular la distribución de manera más realista en los cálculos de tiempo.

2. Donde encontrar documentación y donde pedir ayuda.

2.1. Documentación oficial

Para esta librería y cualquier otra que tengáis que aprender, debéis acostumbraros a leer las cabeceras de código y los manuales oficiales.

[cabeceras de la versión 4.0.3](#)

[cabeceras de la versión 3.2](#)

2.2. Tutoriales

También es normal necesitar ejemplos de código funcionando para poder comparar, entender los procesos y desde ahí, ampliar el conocimiento.

Github y otros repositorios de código están llenos de proyectos que emplean MPICH.

Otro sitio que está genial para empezar es: <https://mpitutorial.com>

Otro con ejemplos de código es un foro oficial del estándar MPI, a priori da igual la versión de MPI que elijáis, pues las funciones que empleamos en la práctica no han cambiado: <https://www.mpi-forum.org/docs>

2.3. Referencias y ayuda

Stackoverflow, en github y demás portales de repositorios de código...

Evitaría el uso de **ChatGPT, Gemini, Claude...**, estáis aprendiendo e ir a por la solución sin acceder a las fuentes primarias de forma crítica no os va a ayudar :-/.

3. Puesta en marcha de un clúster con MPICH usando docker

3.1. Instalación de docker

Necesitaréis tener instalado docker y docker compose (versión 2 o superior), en Ubuntu 22.04 GNU/Linux usaremos estos comandos:

```
sudo apt update
```

```
sudo apt install docker.io docker docker-buildx docker-compose-v2
```

Revisamos los permisos del usuario para usar docker:

```
sudo groupadd docker
```

```
sudo usermod -aG docker $USER
```

Para que estos cambios de permisos se haga efectivo, debemos cerrar y volver a iniciar la sesión.

Podría pasar que el demonio de Docker no esté ejecutando en segundo plano (Ejecutar Docker da error que no se puede conectar con el socket de Docker), para ponerlo en marcha en sistemas GNU/Linux que usen systemd:

```
sudo systemctl enable docker
sudo systemctl start docker
```

Para otros sistemas con OpenRC, GNU Shepard...:

```
service docker start
```

Si nada de esto funciona, ejecutamos el demonio de docker de manera manual en un terminal aparte que tendrá que estar funcionando en todo momento:

```
dockerd
```

3.2. Clonación del repositorio de código con las imágenes de docker para MPICH

Para realizar las prácticas sin necesidad de un clúster real de equipos, vamos a crear un clúster usando docker

Para ellos vamos a usar un repositorio con las imágenes necesarias de Docker usando git:

```
sudo apt update
sudo apt install git
```

El repositorio es <https://github.com/gentooza/alpine-mpich> en su rama específica para nuestras clases de MPICH, 'new':

```
git clone -b new git@github.com:gentooza/alpine-mpich.git
ó si no teneís usuaria/o de Github
git clone -b new https://github.com/gentooza/alpine-mpich.git
```

Esto nos creará una carpeta llamada alpine-mpich donde vamos a trabajar

3.3. Estructura del repositorio

El repositorio consta de 3 carpetas diferenciadas donde:

- **Carpeta base/** Donde hay un Dockerfile que configura una imagen base de Alpine Linux con lo necesario para las prácticas.

- **Carpeta onbuild/** Donde hay un Dockerfile que configura una imagen basada en la nuestra Base, que la complementa en el proceso de construcción con unos scripts necesarios en tiempo de ejecución.
- **Carpeta cluster/** Donde vamos a poner nuestro código fuente en ficheros .c dentro de la carpeta project, hay un Dockerfile donde agregaremos líneas de compilación para compilarlos, y un script cluster.sh al que llamaremos y hará un docker compose del fichero docker-compose.yml que hay y nos levantará el clúster, haremos login en el nodo maestro, etc.

3.4. Construcción de las imágenes base

Para construirnos la imagen base que se va a emplear, que es un Alpine GNU/Linux, en el raíz ejecutamos:

```
docker build --build-arg MPICH_VERSION="4.0.3" -t nlknguyen/alpine-mpich base/
```

OJO al argumento después de -t, debe de ser nlknguyen/alpine-mpich no os equivoquéis, si no, no estaréis sobrescribiendo la imagen original que está obsoleta!

Este proceso puede tardar incluso horas dependiendo de la potencia de la máquina en la que estemos ejecutándolo, pues estamos compilando la librería MPICH entera dentro de nuestra imagen.

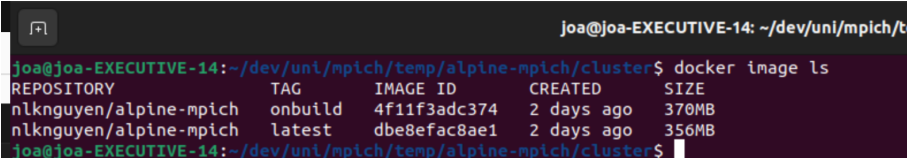
Otra imagen que habrá que crear es la del estado onbuild, ésta se basará en la base que ya hemos creado pero ampliará su funcionalidad con unos scripts necesarios para conocer el clúster donde se va a trabajar, las claves ssh para interconectarse entre nodos...

```
docker build -t nlknguyen/alpine-mpich:onbuild onbuild/
```

Una vez creadas estas imágenes, deben de aparecernos utilizando el comando docker, el comando que debemos utilizar para listarlas es este:

```
docker image ls
```

Un ejemplo de retorno de este comando sería el de la Figura .



```

Joa@Joa-EXECUTIVE-14: ~/dev/unl/mpich/t
Joa@Joa-EXECUTIVE-14:~/dev/unl/mpich/temp/alpine-mpich/cluster$ docker image ls
REPOSITORY          TAG         IMAGE ID      CREATED       SIZE
nlknguyen/alpine-mpich onbuild     4f11f3adc374  2 days ago   370MB
nlknguyen/alpine-mpich latest      dbefac8ae1    2 days ago   356MB
Joa@Joa-EXECUTIVE-14:~/dev/unl/mpich/temp/alpine-mpich/cluster$

```

Figura 2: Captura de terminal de la respuesta del comando 'docker image ls'.

3.5. Puesta en marcha del clúster

Con los pasos hasta ahora tenemos la imagen base hecha, y tenemos la imagen en el estado onbuild preparada para el despliegue con todo lo de la base y los scripts de configuración de la red y SSH.

A continuación levantamos el clúster de trabajo con el conjunto de contenedores virtuales con réplicas de las imágenes creadas. Para ello ejecutamos el script cluster.sh **desde la carpeta clúster:**

```
cd cluster
./cluster.sh up size=8
```

La opción size nos marcará cantidad de nodos que posee nuestro clúster.

Este terminal se deberá dejar en segundo plano mientras lancemos las ejecuciones de la(s) actual(es) aplicación(es) de mpich que tengamos cargada(s). Se verá algo como lo mostrado en la Figura 3

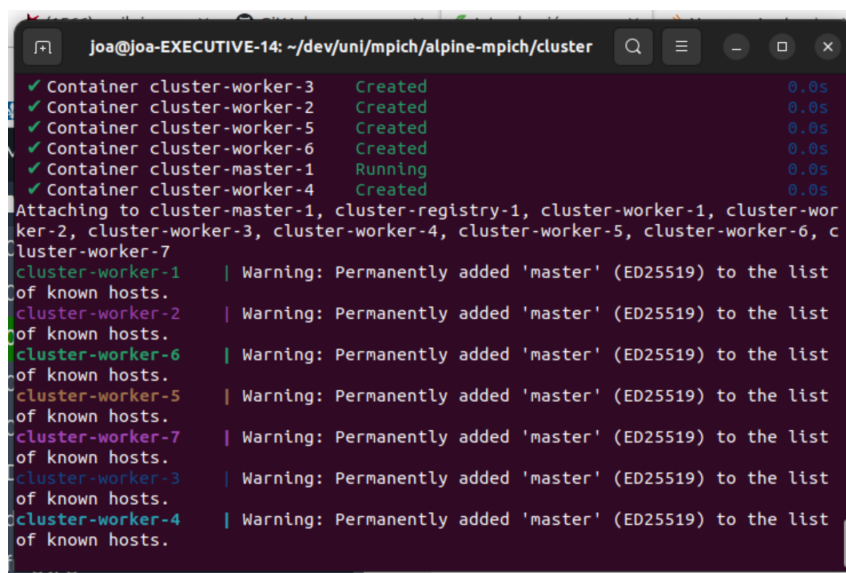


Figura 3: Captura de terminal del clúster corriendo.

Para trabajar con el clúster habrá que entrar en el nodo maestro, **en MPICH la misma aplicación es compilada con la misma versión de MPICH, con el mismo usuario y path en cada una de las máquinas. Pero es el nodo maestro el que lanzará la aplicación, repartirá la carga de trabajo y recogerá los resultados.**

Para entrar en el nodo maestro habrá que utilizar de nuevo el script cluster.sh de la siguiente forma:

```
# debemos estar en la misma carpeta que el script, si no lo estamos:
cd cluster
./cluster.sh login
```

Se muestra como sería de forma real en la Figura 4

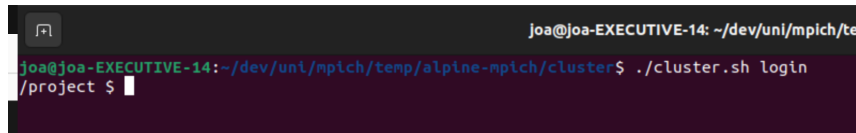


Figura 4: Captura de terminal de cluster.sh login.

4. Estructura básica de una aplicación en C con MPICH

4.1. Agregar la cabecera de la librería

¡Lo primero de lo primero! Habrá que **agregar la cabecera de la librería MPICH** con su include, así podremos utilizar sus funciones

```
#include "mpi.h"
[...]
```

4.2. Iniciar main con argumentos

Esto lo hacemos aunque no se va a usar en la práctica, pero es buena opción tener los argumentos de la función main declarados por si llegado el caso se quieren emplear, así podríamos pasarle parámetros a MPICH en tiempo de ejecución.

```
[...]
int main (int argc, char* argv[])
{
[...]
```

4.3. Iniciar mpich

Antes de emplear las funciones de la librería MPICH deberemos inicializarla. **Antes de esta función no se puede llamar ninguna otra función de MPICH**

Antes de cerrar la ejecución de nuestra aplicación se deberá finalizar la librería MPICH para la correcta liberación de memoria y recursos. **Después de esta función ya no se podrá emplear más funciones de MPICH**

```
[...]
MPI_Init(&argc, &argv);
[...]
```

```
MPI_Finalize();
[...]
```

4.4. Temporización de las prácticas con mpich

Los ejercicios habrá que temporizarlos para poder realizar una comparativa de tiempos de ejecución. Para ellos se empleará la función interna de MPICH: `MPI_Wtime()`

```
double start, end;
start = MPI_time();
[...]
// aquí todo el meollo de cómputo
[...]
end = MPI_time();
printf("Tiempo total de ejecución: %f\n", end - start);
[...]
```

Recordad: solo temporizamos el cómputo, ni el cálculo de las matrices iniciales, ni la impresión de resultados. Y solo realiza el cálculo el nodo maestro, que es el que reparte y recoge los resultados.

4.5. Ejemplo de hola mundo con MPICH

Este es el código que podemos encontrar dentro de `cluster/project/` en nuestro repositorio. Este código simplemente inicializa mpi, obtiene el número de procesos que hay en clúster, obtiene su rango dentro del clúster y sus datos y los imprime.

```
#include <mpi.h>
#include <stdio.h>

int main(int argc, char** argv) {
    // Initialize the MPI environment. The two arguments to MPI Init are not
    // currently used by MPI implementations, but are there in case future
    // implementations might need the arguments.
    MPI_Init(NULL, NULL);

    // Get the number of processes
    int world_size;
    MPI_Comm_size(MPI_COMM_WORLD, &world_size);

    // Get the rank of the process
    int world_rank;
    MPI_Comm_rank(MPI_COMM_WORLD, &world_rank);
```



```

// Get the name of the processor
char processor_name[MPI_MAX_PROCESSOR_NAME];
int name_len;
MPI_Get_processor_name(processor_name, &name_len);

// Print off a hello world message
printf("Hello world from processor %s, rank %d out of %d processors\n",
       processor_name, world_rank, world_size);

// Finalize the MPI environment. No more MPI calls can be made after this
MPI_Finalize();
}

```

Como este código se va a ejecutar en cada nodo, cada uno va a realizar estas tareas y las va a imprimir.

4.6. Compilación y ejecución del hola mundo en nuestro clúster Docker

Tenemos que tener la precaución de que el mismo código se compile y esté almacenado en el mismo path, mismo usuario, en cada uno de los contenedores del clúster. Para ello tenemos que partir del mismo código almacenado dentro de cluster/project como se muestra en la Figura 5 y debemos declarar su compilación mediante el compilador mpicc dentro de archivo cluster/Dockerfile como se muestra en la Figura 6

Una vez creado nuestro código en C, y preparado para compilar. Para que aparezca en nuestro clúster, deberemos tirarlo abajo (cerrar el terminal donde se ejecutaba o pulsar ctrl +c para cerrarlo) y volver a ejecutarlo con ./cluster.sh up size=8. 3

Para ejecutarlo necesitaremos entrar en el nodo maestro una vez levantado el clúster. 4

Para la ejecución de las aplicaciones MPICH se necesita declarar lo que comúnmente se denomina machinefile o un archivo de hosts, es un fichero en texto plano que nos dará los nombres o IPs de red de los equipos que conforman el clúster, incluido nuestro nodo maestro. Más el número de procesos que ejecuta cada nodo, si no se pone nada es 1 proceso.

Para obtener las direcciones de los equipos del clúster, ejecutaremos el script get_hosts que viene incluido en nuestro sistema gracias a la imagen onbuild que construimos. La salida del script la guardaremos en un archivo de texto plano que llamaremos machinefile de esta forma:

```
get_hosts > machinefile
```

Podéis observar la ejecución en acción en la Figura 7:

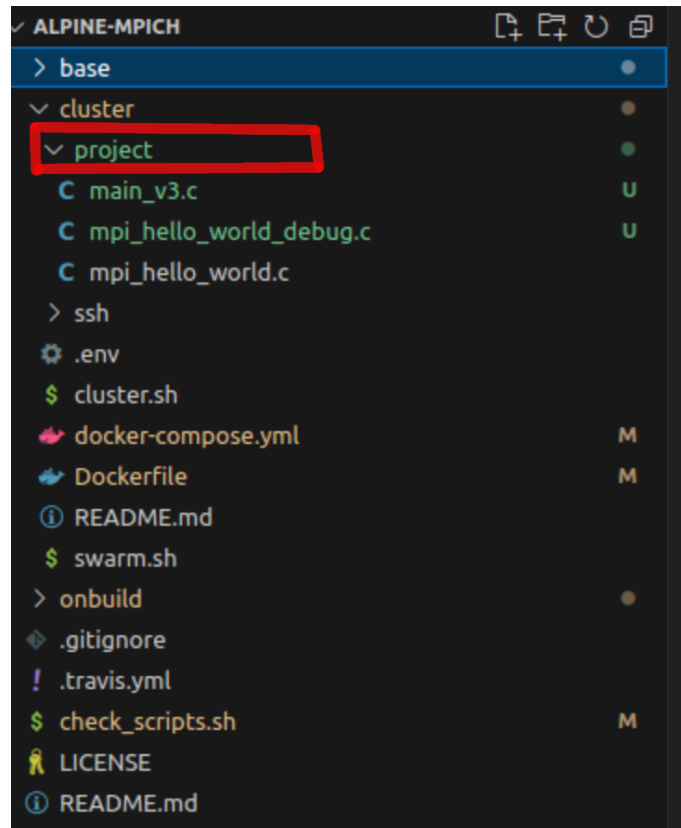


Figura 5: Guardamos nuestro .c en la carpeta project dentro de cluster.

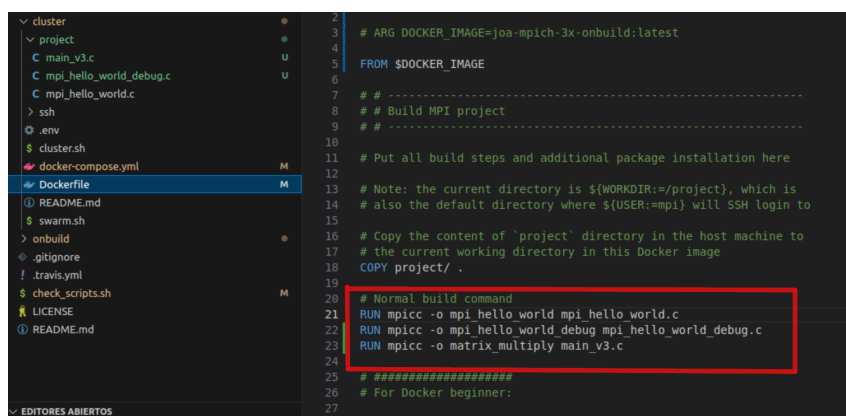
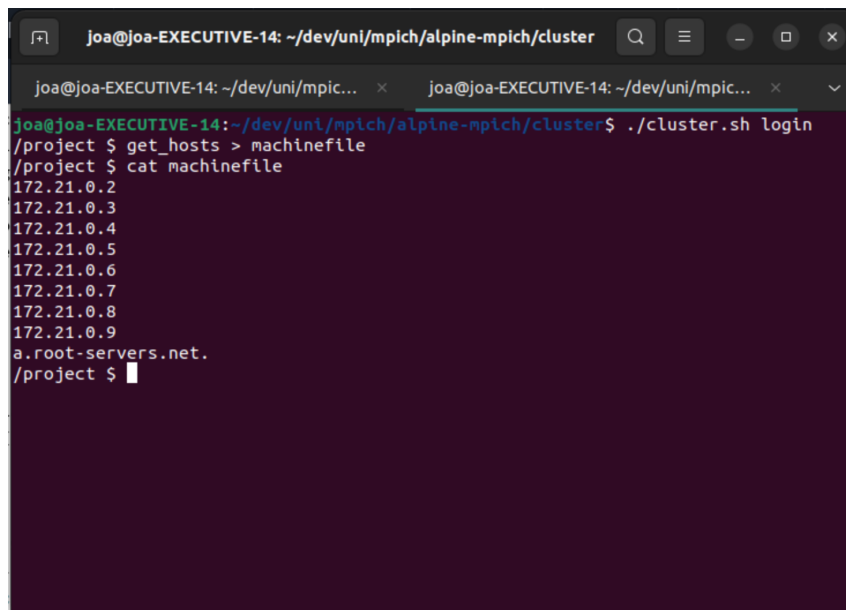


Figura 6: Añadir la línea de ejecución de mpicc en nuestro dockerfile del clúster, para compilar nuestra aplicación.



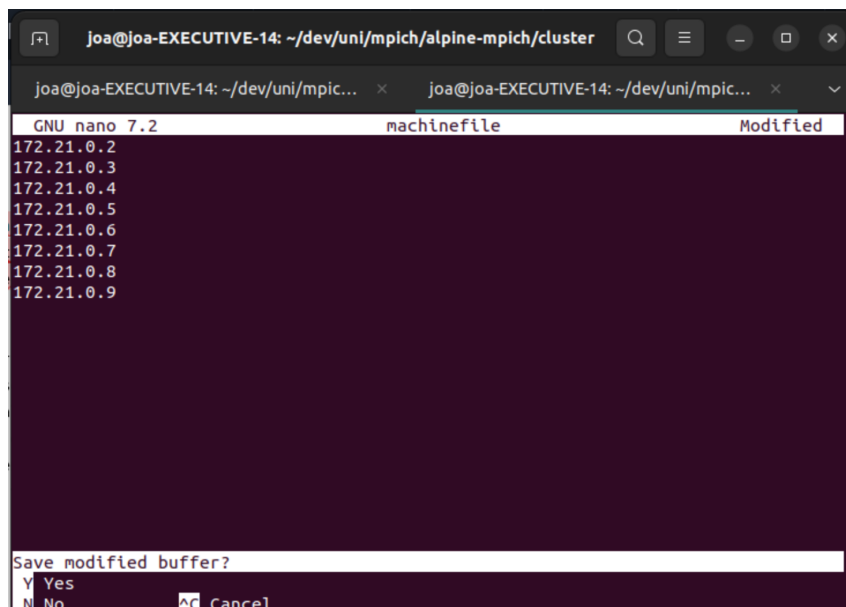
```
joa@joa-EXECUTIVE-14: ~/dev/uni/mpich/alpine-mpich/cluster
joa@joa-EXECUTIVE-14: ~/dev/uni/mpic... x joa@joa-EXECUTIVE-14: ~/dev/uni/mpic... x
joa@joa-EXECUTIVE-14:~/dev/uni/mpich/alpine-mpich/cluster$ ./cluster.sh login
/project $ get_hosts > machinefile
/project $ cat machinefile
172.21.0.2
172.21.0.3
172.21.0.4
172.21.0.5
172.21.0.6
172.21.0.7
172.21.0.8
172.21.0.9
a.root-servers.net.
/project $
```

Figura 7: Ejemplo salvando `get_hosts` en `machinefile` y mostrando el contenido del fichero.

Hay un error en el script `get_hosts` por el que inserta la línea `a.root.server.net` al final, esa línea debe ser eliminada para poder ejecutar la aplicación. Para ellos tenéis a vuestra disposición el editor de texto por consola, `nano`:

```
nano machinefile
```

Borráis esa línea y luego salváis de nuevo el fichero con un `ctrl + x` y diciendo que sí y guardando el archivo con el mismo nombre, como en las Figuras 8 y 9.



```
GNU nano 7.2 machinefile Modified
172.21.0.2
172.21.0.3
172.21.0.4
172.21.0.5
172.21.0.6
172.21.0.7
172.21.0.8
172.21.0.9
Save modified buffer?
Y Yes
N No ^C Cancel
```

Figura 8: Eliminamos la línea y presionamos `ctrl + x`, decimos sí (una Y).

Este `machinefile` tal como está nos declara 8 nodos de computación con sólo un proceso cada uno.

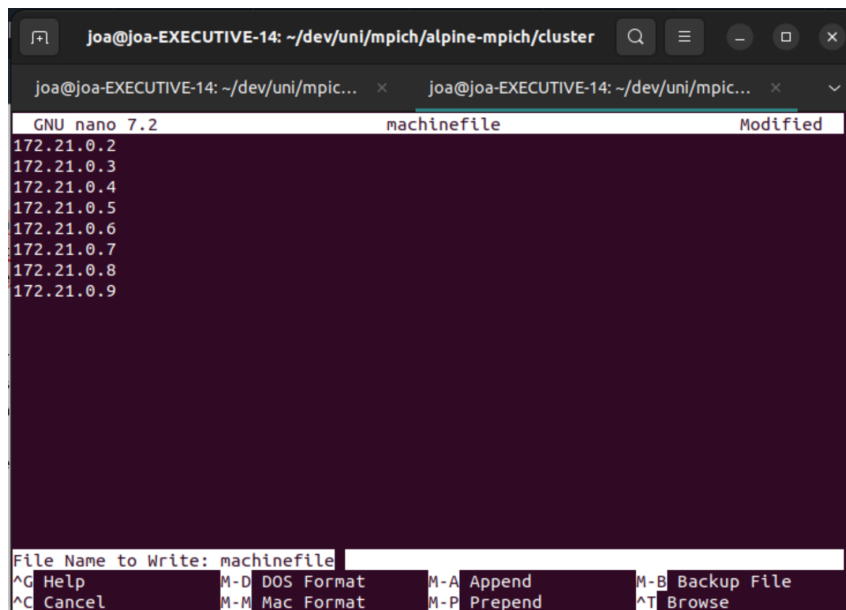


Figura 9: Salvamos los cambios en machinefile igual.

Para ejecutar el código de hola mundo usaremos el comando `mpiexec` (o `mpirun`) de la siguiente forma:

```
mpiexec -f ./machinefile ./mpi_hello_world
```

Esto ejecutará el código hola mundo en cada una de las máquinas del clúster gracias al `machinefile` suministrado.

Podéis ver el resultado de la Figura. [10](#)

5. Desarrollo local con MPICH. Como probar nuestro código de manera ágil?

Docker está genial y nuestro clúster funciona bien para tener un conjunto de equipos con MPICH funcionando, con recursos de CPU y RAM limitados, y con interconexión entre ellos.

Pero tiene un problema! Para compilar nuevas versiones de nuestro código hay que tirar abajo el clúster y volver a levantarlo.

Esto no parece muy ágil, sobre todo cuando estamos aún intentando que nuestro código compile o comprobando que haga el algoritmo que queremos que realice.

Para solucionar este problema, y teniendo GNU/Linux, podéis realizar una instalación local de MPICH, en el caso de Ubuntu/ Linux Mint/ Debian... de esta forma:

```
sudo apt install mpich
```

Con MPICH instalado localmente podremos compilar el código que estemos desarrollando mediante:

```

/project $ get_hosts > machinefile
/project $ nano machinefile
/project $ mpxexec -f ./machinefile ./mpi_hello_world
hwloc/linux: Ignoring PCI device with non-16bit domain.
Pass --enable-32bits-pci-domain to configure to support such devices
(warning: it would break the library ABI, don't enable unless really needed).
hwloc/linux: Ignoring PCI device with non-16bit domain.
Pass --enable-32bits-pci-domain to configure to support such devices
(warning: it would break the library ABI, don't enable unless really needed).
hwloc/linux: Ignoring PCI device with non-16bit domain.
Pass --enable-32bits-pci-domain to configure to support such devices
(warning: it would break the library ABI, don't enable unless really needed).
hwloc/linux: Ignoring PCI device with non-16bit domain.
Pass --enable-32bits-pci-domain to configure to support such devices
(warning: it would break the library ABI, don't enable unless really needed).
hwloc/linux: Ignoring PCI device with non-16bit domain.
Pass --enable-32bits-pci-domain to configure to support such devices
(warning: it would break the library ABI, don't enable unless really needed).
hwloc/linux: Ignoring PCI device with non-16bit domain.
Pass --enable-32bits-pci-domain to configure to support such devices
(warning: it would break the library ABI, don't enable unless really needed).
Hello world from processor cbee31d5c014, rank 0 out of 8 processors
Hello world from processor cd9ffd3bcb63, rank 4 out of 8 processors
Hello world from processor 313f0371406a, rank 3 out of 8 processors
Hello world from processor 0465a2dfc027, rank 5 out of 8 processors
Hello world from processor 82b8cd8820b3, rank 1 out of 8 processors
Hello world from processor a8f33e336633, rank 6 out of 8 processors
Hello world from processor 29de76cec6b3, rank 2 out of 8 processors
Hello world from processor 01d9d3929771, rank 7 out of 8 processors
/project $

```

Figura 10: Captura de los pasos y el resultado de la ejecución del ejemplo hola mundo.

```
mpicc -o <mi ejecutable> ./<mi código en c>
```

por ejemplo:

```
mpicc -o test ./main.c
```

A su vez lo podremos ejecutar incluso simulando varias máquinas con la opción `-np` de la forma:

```
# para ejecutar con 4 procesos
```

```
mpirun -np 4 ./test
```